

MSAI605: Project Milestone 4 System Card

Authors: Pramod Goyal, Arun Kulkarni

Course: MSAI605

Milestone 4: System Card

System version: milestone4_final

Planned release tag: v1.0-final

Operating threshold: 0.3969849246231156 (selected on the validation split via balanced accuracy)

Test split source: outputs/pairs_test.npz, evaluated under run_07 in outputs/runs_log.json

Source artifacts: - Final system summary: outputs/final_system_summary.json - Inference config: configs/inference_config.json - CLI entrypoint: scripts/verify.py - Companion profiling report: reports/milestone4_profiling_report.pdf - Reproducibility checklist: reports/milestone4_reproducibility_checklist.md

1. System overview

This project is a face verification system for deciding whether two already-cropped face images belong to the same person. The released pipeline accepts two input images, resizes each to 160 x 160, normalizes pixels to [-1, 1], extracts 512-dimensional FaceNet embeddings with InceptionResnetV1 pretrained on VGGFace2, L2-normalizes both embeddings, and computes cosine similarity. The final operating threshold is 0.3969849246231156; scores at or above that threshold are labeled SAME, otherwise DIFFERENT.

The CLI entrypoint is scripts/verify.py, which reports the similarity score, threshold, binary decision, confidence value, and latency breakdown for each evaluated pair. Confidence is computed as $\text{sigmoid}(10 * (\text{score} - \text{threshold}))$, which means it is a smooth function of distance from the decision boundary rather than a calibrated probability of identity match.

2. Intended use

This project is intended for course demonstration, reproducible experimentation, and responsible reporting of an embedding-based face verification workflow on a benchmark dataset. Appropriate uses include:

- reproducing the course milestone deliverables
- inspecting model behavior on the LFW-style verification task

- comparing thresholded verification behavior under a fixed embedding model and release configuration

Out-of-scope uses include:

- identity verification in safety-critical or high-stakes settings
- surveillance, tracking, or automated law-enforcement workflows
- demographic or subgroup fairness claims beyond what the available project data can support
- unconstrained production deployment where face detection, alignment robustness, enrollment policy, and security controls are required

3. Data summary

Development and evaluation use Labeled Faces in the Wild (LFW) pair splits. The project operates on pre-generated image pairs and reports results on the project's validation and test pair files. The final threshold was selected on the validation split (run_06) using the `balanced_acc` rule and then evaluated once on the held-out test split (run_07).

Important data limitations:

- LFW is a benchmark dataset, not a deployment dataset, so observed performance should be interpreted as benchmark performance only.
- The pipeline expects already-cropped faces and does not evaluate missed detections, bad crops, or multi-face scenes.
- The project does not maintain reliable demographic metadata for subgroup analysis, so fairness discussion is limited to qualitative risk framing rather than subgroup claims.

4. Operating threshold and metrics

The final release threshold is 0.3969849246231156, selected on the validation split with balanced accuracy as the operating rule. The final test run uses the same threshold and reports:

Metric	Value
Balanced accuracy	0.980
F1	0.980
Equal error rate	0.024
TP	489

FP	9
TN	491
FN	11

These values come from `outputs/runs_log.json`, specifically the embedding sweep `run_06` and embedding final test run `run_07`. All release-facing artifacts cite this same threshold and these same test metrics, and `scripts/validate_release_alignment.py` enforces that consistency before tagging.

5. Failure modes and limitations

The most important operational limitations come from the fact that this system compares fixed embeddings under a single global threshold:

- visually similar different-identity pairs can score above threshold and produce false accepts
- pose, lighting, blur, occlusion, and expression shifts can reduce similarity for same-identity pairs and produce false rejects
- scores near the threshold have inherently higher decision fragility, so small image perturbations can flip the binary decision
- the system assumes the input image already contains a usable face crop and does not detect invalid framing, spoofing, or tampering

Because the confidence value is a transformed margin from the threshold, users should not interpret a value like `0.90` as "90% probability these faces match." It only means the score lies farther from the configured decision boundary in the positive direction.

6. Fairness-related risks and misuse concerns

Face verification systems can produce uneven error patterns across demographic groups, capture conditions, and collection contexts. This project does not include reliable demographic annotations for the evaluation data, so it cannot support quantitative subgroup fairness claims. The appropriate responsible statement is that fairness-related risk exists, but the project has not established evidence to quantify or dismiss it.

Misuse concerns include:

- using the model to make consequential identity judgments about people without human review
 - applying the threshold outside the benchmark setting without local validation
 - treating benchmark accuracy as proof of suitability for public-facing or institutional deployment
-

7. Operational constraints

The released configuration is CPU-oriented and documented in `configs/inference_config.json`. Input assumptions are:

- two face images are provided explicitly
- images are valid RGB-compatible arrays or files readable by the CLI
- no face detection or alignment search is performed inside the released verifier

For runtime characteristics, see `reports/milestone4_profiling_report.pdf`. That report documents the CPU-only baseline, stage-wise latency breakdown, and batch-size sensitivity. At a high level, embedding inference dominates latency (>93% of end-to-end at every tested batch size on the reported hardware), while preprocessing and cosine-scoring are comparatively small.

8. Reproducibility pointer

Release-aligned artifact locations:

- README: `README.md`
- profiling report: `reports/milestone4_profiling_report.pdf`
- reproducibility checklist: `reports/milestone4_reproducibility_checklist.md`
- final system summary: `outputs/final_system_summary.json`
- final tag name: `v1.0-final`

The helper script `scripts/validate_release_alignment.py` checks that the config, run log, summary, and final report paths are aligned before tagging. The companion test `tests/test_final_release_alignment.py` is run as part of `pytest tests/` and additionally exercises the validator against synthetic consistent and inconsistent contracts so regressions in the release-alignment check itself are caught.